

FAKE NEWS DETECTION SYSTEM

CodeVedX Internship Project

Submitted By

Name: Aarti Prajapati

Enrollment Number: 21231258

Course: B.Tech CSE (AI & ML), 7th Semester

College: IIMT University, Meerut ,Uttar Pradesh

Email : aartiaiml3009@gmail.com

Project Theme

The project focuses on applying Machine Learning and Natural Language Processing (NLP) techniques to automatically classify news articles as **Fake** or **Real**. It aims to reduce the spread of misinformation by providing a fast and accurate news verification system.

GitHub Repository

https://github.com/aartiprajapati-ai/CODEVEDX/tree/main/Fake_News_Detection_System

Academic Year: 2026–2027

INTRODUCTION

The **Fake News Detection System** is a Machine Learning-based application developed to identify whether a news article is **Fake** or **Real**. With the rapid growth of social media and online news platforms, the spread of false information has become a major challenge. Fake news can influence public opinion, create confusion, and spread misinformation within a very short time.

This project uses **Natural Language Processing (NLP)** and **Machine Learning** techniques to analyze the textual content of news articles. The text is preprocessed, converted into numerical features using the **TF-IDF Vectorizer**, and classified using the **Logistic Regression** algorithm. The trained model predicts whether the input news is fake or genuine.

A simple and interactive **Streamlit** web application is developed to allow users to enter news text and instantly receive the prediction. This project demonstrates the practical implementation of Machine Learning, text preprocessing, feature extraction, model training, and deployment in solving a real-world problem.

The main objective of this project is to build an efficient, user-friendly, and reliable system that helps users identify fake news and promotes awareness about misinformation in the digital world.

PROBLEM STATEMENT

With the rapid growth of the internet and social media platforms, fake news spreads faster than ever before. Misleading or false information can influence public opinion, create confusion, and negatively impact society. Manually verifying every news article is difficult, time-consuming, and often impractical due to the huge volume of information published daily.

The challenge is to develop an intelligent system that can automatically analyze the content of a news article and determine whether it is **Fake** or **Real**. Such a system should provide accurate and quick predictions while reducing the need for manual verification.

This project addresses this problem by applying **Machine Learning** and **Natural Language Processing (NLP)** techniques to classify news articles based on their textual content. The goal is to build a reliable and user-friendly Fake News Detection System that helps users identify misinformation efficiently.

PROJECT OBJECTIVE

The main objectives of the **Fake News Detection System** are:

- To develop a Machine Learning model that can accurately classify news articles as **Fake** or **Real**.
- To apply **Natural Language Processing (NLP)** techniques for preprocessing and analyzing textual news data.
- To convert textual data into numerical features using the **TF-IDF Vectorizer** for effective model training.
- To train and evaluate a **Logistic Regression** model for news classification.
- To build a simple and user-friendly **Streamlit** web application for real-time news prediction.
- To reduce the spread of misinformation by providing quick and reliable news verification.
- To enhance practical knowledge of Machine Learning, NLP, data preprocessing, feature engineering, model training, and deployment.
- To create a reusable and scalable solution that can be improved with advanced Machine Learning and Deep Learning techniques in the future.

EXPECTED OUTCOMES

The expected outcomes of the **Fake News Detection System** are:

- To accurately classify news articles as **Fake** or **Real** using Machine Learning techniques.
- To reduce the spread of misinformation by providing quick and reliable predictions.
- To develop a user-friendly web application using **Streamlit** for easy interaction.
- To improve understanding and practical implementation of **Natural Language Processing (NLP)** and **Machine Learning** concepts.
- To preprocess textual data efficiently using **TF-IDF Vectorization** for better model performance.
- To build a reusable trained model that can classify new and unseen news articles.
- To provide fast and reliable predictions with high accuracy.
- To create a project that demonstrates real-world applications of Artificial Intelligence and Machine Learning.

SCOPE OF THE PROJECT

The scope of the **Fake News Detection System** is to develop an intelligent application that automatically classifies news articles as **Fake** or **Real** using Machine Learning and Natural Language Processing (NLP) techniques. The system is designed to assist users in identifying potentially misleading news quickly and efficiently.

The project focuses on text-based news classification by preprocessing news content, extracting meaningful features using the **TF-IDF Vectorizer**, and applying the **Logistic Regression** algorithm for prediction. A simple **Streamlit** web application is provided to enable users to enter news text and receive instant prediction results.

This project can be further expanded by integrating real-time news APIs, multilingual support, advanced Deep Learning models such as **LSTM** or **BERT**, and cloud deployment for wider accessibility. It can also be adapted for use by media organizations, educational institutions, fact-checking platforms, and digital news applications to help reduce the spread of misinformation.

The project serves as a practical demonstration of applying Artificial Intelligence and Machine Learning to solve real-world problems related to digital media and information verification

LIMITATIONS

The **Fake News Detection System** has the following limitations:

- The model's prediction accuracy depends on the quality and size of the training dataset.
- It can classify only **text-based news** and does not analyze images, videos, or audio content.
- The system may not perform well on completely new topics or news written in a different style than the training data.
- It currently supports only **English-language** news articles.
- The model predicts based on patterns learned from historical data and cannot verify the factual correctness of news in real time.
- Short or incomplete news articles may lead to less accurate predictions.
- The application does not use live news APIs or real-time fact-checking databases.
- The project uses **Logistic Regression**, which is efficient and accurate but may be outperformed by advanced Deep Learning models such as **BERT** or **LSTM** on more complex datasets.

SOFTWARE REQUIREMENTS

The following software and tools were used to develop the **Fake News Detection System**:

Software/Tool	Purpose
Operating System	Windows 10/11 or Linux
Programming Language	Python 3.x
IDE/Code Editor	Visual Studio Code (VS Code)
Notebook Environment	Jupyter Notebook / Google Colab
Framework	Streamlit
Libraries	Pandas, NumPy, Scikit-learn, NLTK, Pickle
Version Control	Git & GitHub
Dataset Format	CSV Files (Fake.csv, True.csv)

HARDWARE REQUIREMENTS

Hardware Requirements

The following hardware configuration is sufficient to develop and run the **Fake News Detection System**:

Hardware Component	Minimum Requirement
Processor	Intel Core i3 / AMD Ryzen 3 or above
RAM	4 GB (8 GB Recommended)
Storage	Minimum 5 GB Free Disk Space
Display	1366 × 768 Resolution or Higher
Internet Connection	Required for downloading datasets, installing libraries, and GitHub access

Recommended Hardware

- **Processor:** Intel Core i5 / AMD Ryzen 5 or higher
- **RAM:** 8 GB or above
- **Storage:** SSD with at least 10 GB free space
- **Operating System:** Windows 10/11, Ubuntu Linux, or macOS

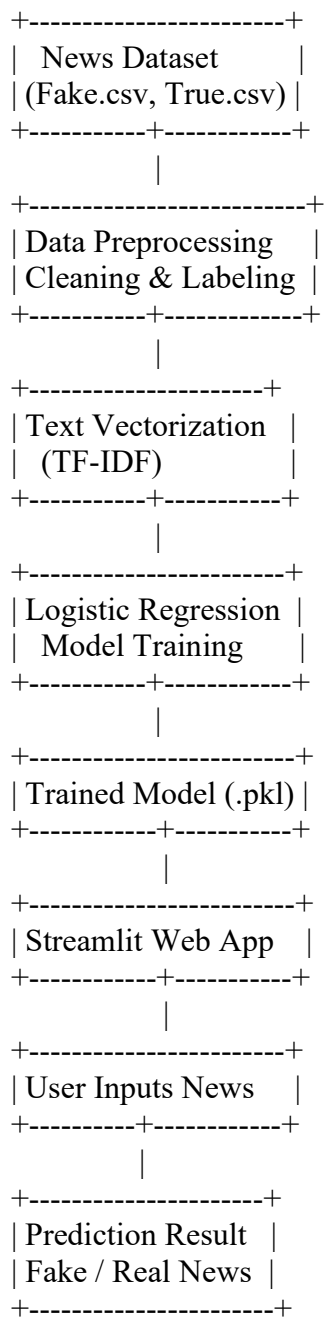
PYTHON LIBRARIES USED

- **Pandas** – Data loading and preprocessing
- **NumPy** – Numerical computations
- **Scikit-learn** – Machine Learning model development
- **NLTK** – Natural Language Processing (text preprocessing)
- **Pickle** – Saving and loading the trained model
- **Streamlit** – Developing the web application

SYSTEM ARCHITECTURE

The **Fake News Detection System** follows a structured Machine Learning pipeline to classify news articles as **Fake** or **Real**. The system processes the input news text through multiple stages, including data preprocessing, feature extraction, model prediction, and result generation.

System Architecture Flow



WORKING OF THE SYSTEM

The **Fake News Detection System** works by processing the input news text through a series of Machine Learning and Natural Language Processing (NLP) steps. The system analyzes the content of the news article and predicts whether it is **Fake** or **Real**.

Step-by-Step Working

Step 1: Data Collection

- The system uses two datasets: **Fake.csv** and **True.csv**.
- Both datasets contain labeled news articles used for training the model.

Step 2: Data Preprocessing

- Missing or unnecessary data is removed.
- The fake and real news datasets are merged.
- Labels are assigned to each news article.
- The text is cleaned and prepared for processing.

Step 3: Feature Extraction

- The cleaned news text is converted into numerical features using the **TF-IDF (Term Frequency–Inverse Document Frequency) Vectorizer**.
- This process helps the Machine Learning model understand the importance of words in each news article.

Step 4: Model Training

- The processed data is divided into training and testing datasets.
- A **Logistic Regression** algorithm is trained using the training data.
- The trained model learns to distinguish between fake and real news based on text patterns.

Step 5: Model Saving

- After successful training, the model and the TF-IDF vectorizer are saved using **Pickle (.pkl)** files.
- These saved files are later used for making predictions without retraining the model.

Step 6: User Input

- The user enters the news article into the **Streamlit** web application.
- The system accepts the text as input for prediction.

Step 7: Prediction

- The input news is preprocessed and transformed using the saved TF-IDF vectorizer.
- The trained Logistic Regression model analyzes the text and predicts whether the news is **Fake** or **Real**.

Step 8: Display Result

- The prediction result is displayed on the Streamlit interface.
- The user can instantly see whether the entered news article is classified as **Fake News** or **Real News**.

DATASET DESCRIPTION

The **Fake News Detection System** uses a publicly available news dataset containing both fake and real news articles. The dataset is used to train and evaluate the Machine Learning model for classifying news as **Fake** or **Real**.

The project uses the following two CSV files:

- **Fake.csv** – Contains fake news articles.
- **True.csv** – Contains genuine (real) news articles.

Both datasets were combined into a single dataset and labeled before preprocessing and model training.

Dataset Features

Column Name	Description
title	Title of the news article
text	Complete content of the news article
subject	Category or topic of the news
date	Publication date of the news article
label	Target variable (0 = Fake News, 1 = Real News)

DATA PREPROCESSING

Before training the model, the dataset was preprocessed using the following steps:

- Merged **Fake.csv** and **True.csv** into a single dataset.
- Assigned labels to fake and real news articles.
- Removed unnecessary columns.
- Cleaned and organized the text data.
- Converted text into numerical features using the **TF-IDF Vectorizer**.

METHODOLOGY

The **Fake News Detection System** was developed using a structured Machine Learning methodology to classify news articles as **Fake** or **Real**. The project follows a sequence of data preprocessing, feature extraction, model training, evaluation, and deployment.

Step 1: Data Collection

- The dataset was collected in CSV format.
- Two files, **Fake.csv** and **True.csv**, were used.
- Both datasets contain news articles with their respective labels.

Step 2: Data Preprocessing

- The two datasets were merged into a single dataset.
- Labels were assigned to identify fake and real news.
- Unnecessary columns were removed.

- Missing values were checked and handled.
- The dataset was cleaned and prepared for model training.

Step 3: Feature Extraction

- The textual data was converted into numerical form using the **TF-IDF (Term Frequency–Inverse Document Frequency) Vectorizer**.
- This technique assigns weights to important words and creates feature vectors for machine learning.

Step 4: Model Training

- The processed dataset was divided into **training** and **testing** sets.
- A **Logistic Regression** algorithm was selected because it is efficient and performs well for text classification tasks.
- The model was trained using the training dataset.

Step 5: Model Evaluation

- The trained model was evaluated using the testing dataset.
- Model performance was measured based on prediction accuracy.
- The project achieved an accuracy of approximately **98.7%**.

Step 6: Model Saving

- The trained model and the TF-IDF vectorizer were saved using the **Pickle (.pkl)** format.
- This allows the model to be reused without retraining.

Step 7: Deployment

- A **Streamlit** web application was developed.
- Users can enter a news article into the application.
- The system processes the input and predicts whether the news is **Fake** or **Real**.

PROJECT IMPLEMENTATION

The **Fake News Detection System** was implemented using Python and Machine Learning techniques to classify news articles as **Fake** or **Real**. The implementation was carried out in a systematic manner, starting from data collection to model deployment.

1. Data Collection

- The project uses two datasets:
 - **Fake.csv**
 - **True.csv**
- Both datasets contain news articles along with their titles, text, subjects, and publication dates.

2. Data Preprocessing

- The fake and real news datasets were merged into a single dataset.

- Labels were assigned to each record (Fake = 0, Real = 1).
- Missing values and unnecessary columns were removed.
- The text data was cleaned and prepared for machine learning.

3. Feature Extraction

- The news text was converted into numerical features using the **TF-IDF (Term Frequency–Inverse Document Frequency) Vectorizer**.
- This process transformed textual data into a format suitable for the machine learning model.

4. Model Training

- The processed dataset was divided into training and testing datasets.
- A **Logistic Regression** algorithm was used to train the classification model.
- The model learned patterns from the news articles to distinguish between fake and real news.

5. Model Evaluation

- The trained model was tested using the testing dataset.
- The prediction accuracy was evaluated to measure the model's performance.
- The model achieved an accuracy of approximately **98.7%**, demonstrating reliable classification performance.

6. Model Saving

- The trained **Logistic Regression** model and the **TF-IDF Vectorizer** were saved using the **Pickle (.pkl)** format.
- These saved files allow predictions to be made without retraining the model.

7. Streamlit Web Application

- A web interface was developed using **Streamlit**.
- Users can enter a news article into the text box.
- After clicking the **Predict** button, the application preprocesses the input, applies the saved TF-IDF vectorizer, and uses the trained model to classify the news as **Fake** or **Real**.
- The prediction result is displayed instantly on the screen.

RESULTS AND OUTPUT

The **Fake News Detection System** was successfully developed and tested using Machine Learning and Natural Language Processing (NLP) techniques. The trained **Logistic Regression** model effectively classified news articles as **Fake** or **Real** based on their textual content. The application was deployed using **Streamlit**, providing a simple and interactive interface where users can enter a news article and receive an instant prediction. The model demonstrated high classification performance, achieving an accuracy of approximately **98.7%** on the test dataset.

Project Results

- Successfully classified news articles into **Fake** and **Real** categories.
- Achieved approximately **98.7% prediction accuracy**.
- Processed news text using **TF-IDF Vectorization**.
- Deployed the trained model as a **Streamlit web application**.
- Generated fast and reliable prediction results.

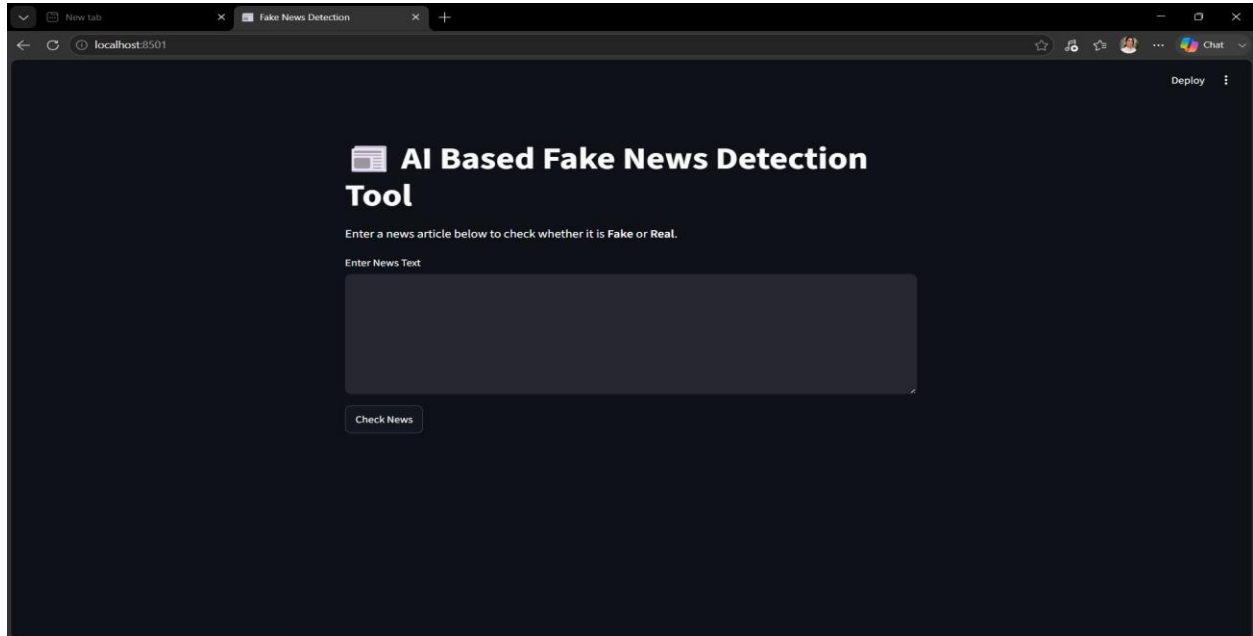
Output Screenshots

1. Home Page

Description:

The Streamlit application provides a simple interface where users can enter a news article and click the **Predict** button to classify the news as **Fake** or **Real**.

Screenshot:

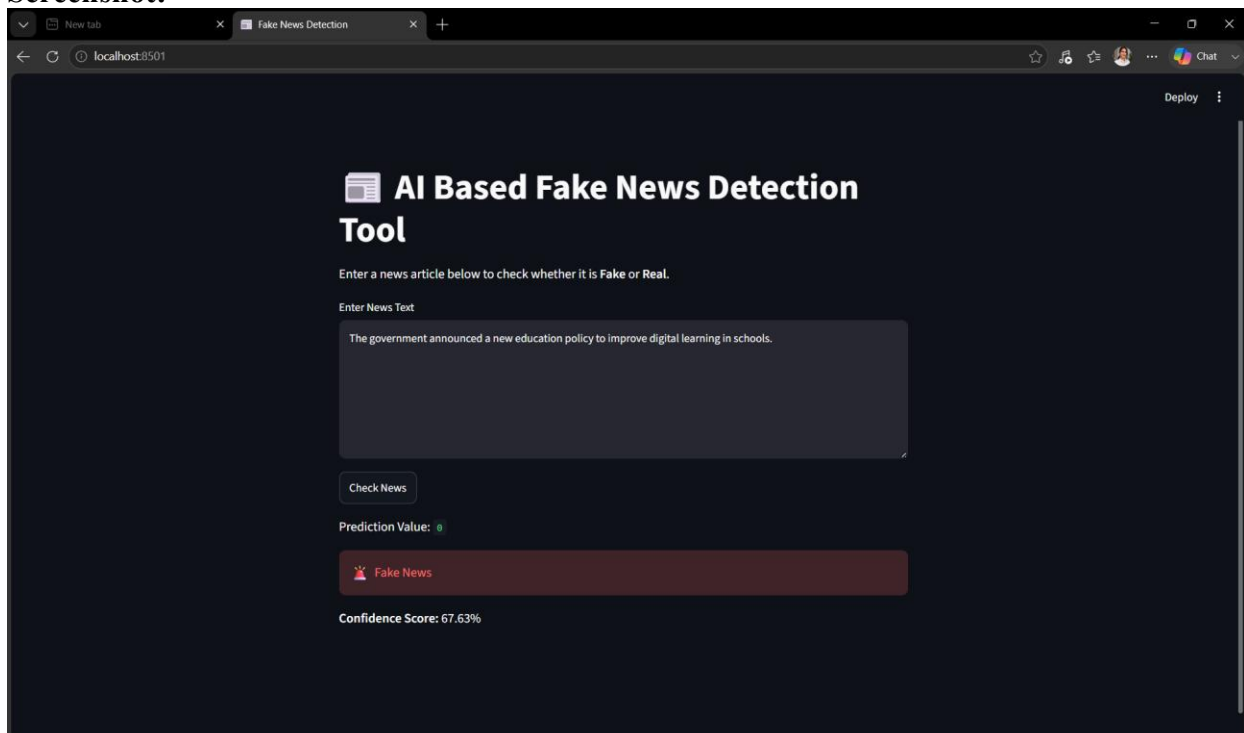


2. Fake News Prediction Result

Description:

The system successfully classified the entered news article as **Fake News** using the trained Logistic Regression model.

Screenshot:

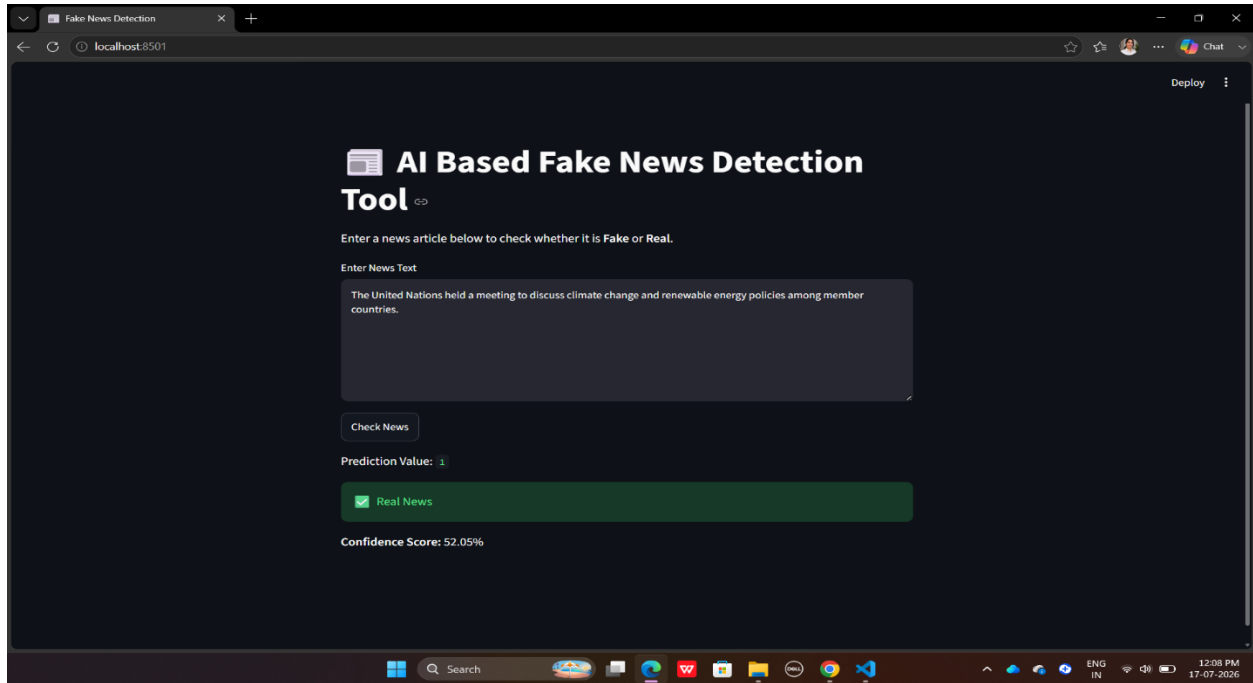


3. Real News Prediction Result

Description:

The system successfully classified the entered news article as **Real News** using the trained Logistic Regression model.

Screenshot:

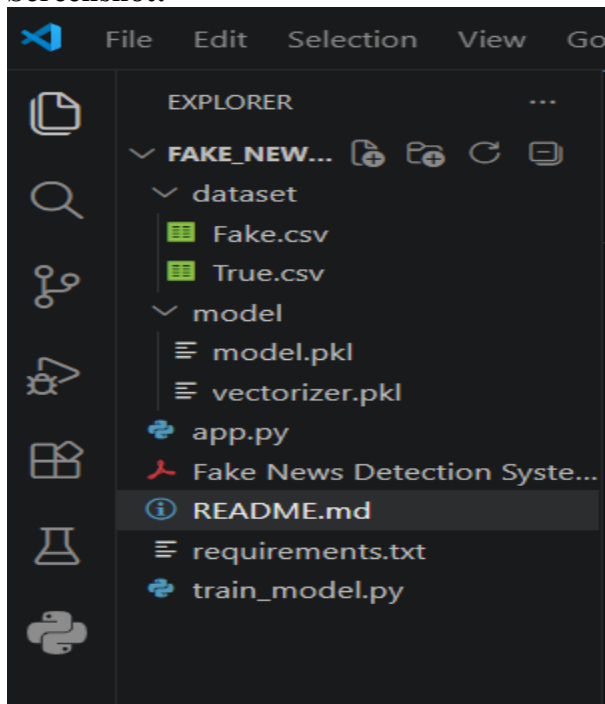


4. Project Structure

Description:

The project follows a well-organized folder structure, making it easy to understand, maintain, and extend.

Screenshot:

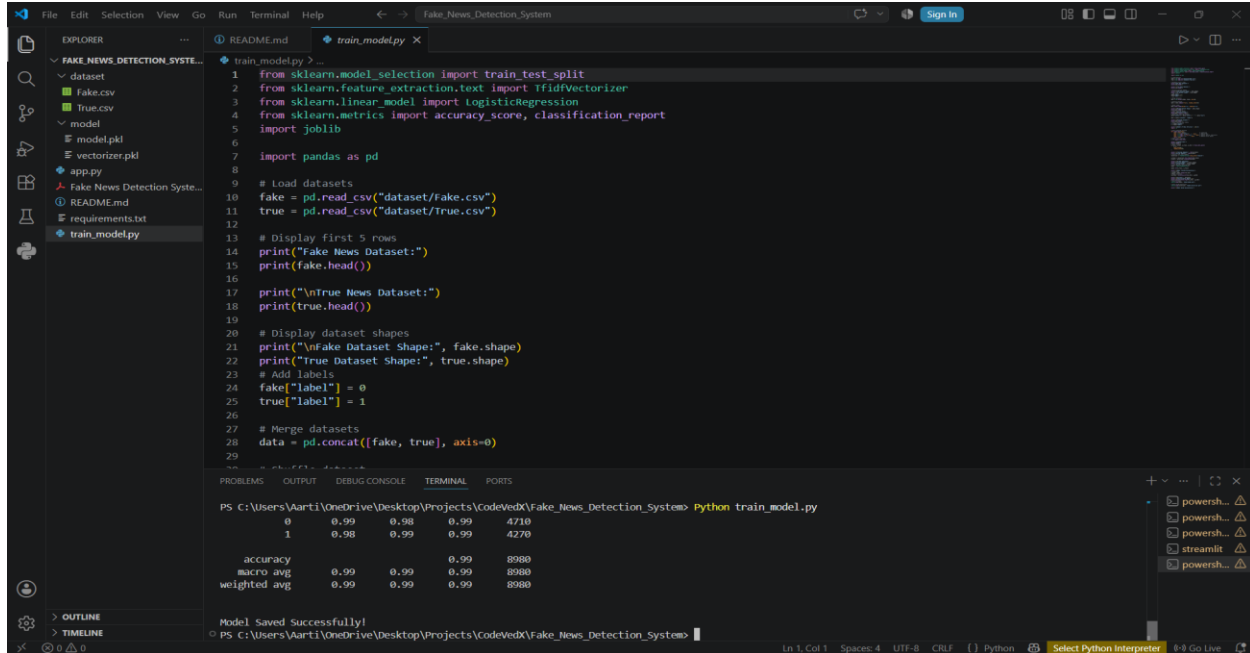


5. Model Training

Description:

The Machine Learning model was trained using the TF-IDF Vectorizer and Logistic Regression algorithm, and the trained model was saved using Pickle library.

Screenshot:



```
1 from sklearn.model_selection import train_test_split
2 from sklearn.feature_extraction.text import TfidfVectorizer
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.metrics import accuracy_score, classification_report
5 import joblib
6
7 import pandas as pd
8
9 # Load datasets
10 fake = pd.read_csv("dataset/fake.csv")
11 true = pd.read_csv("dataset/true.csv")
12
13 # Display first 5 rows
14 print("Fake News Dataset:")
15 print(fake.head())
16
17 print("\nTrue News Dataset:")
18 print(true.head())
19
20 # Display dataset shapes
21 print("\nFake Dataset Shape:", fake.shape)
22 print("\nTrue Dataset Shape:", true.shape)
23
24 # Add labels
25 fake["label"] = 0
26 true["label"] = 1
27
28 # Merge datasets
29 data = pd.concat([fake, true], axis=0)
```

Model Saved Successfully!

	0.99	0.98	0.99	4710
0	0.99	0.98	0.99	4710
1	0.98	0.99	0.99	4270
accuracy			0.99	8980
micro avg	0.99	0.99	0.99	8980
weighted avg	0.99	0.99	0.99	8980

ADVANTAGES

The **Fake News Detection System** offers several benefits, making it a useful application of Machine Learning and Natural Language Processing (NLP).

- Provides **quick and accurate** classification of news articles as **Fake** or **Real**.
- Reduces the time and effort required for manual news verification.
- Uses **Machine Learning** and **Natural Language Processing (NLP)** to analyze textual content efficiently.
- Offers a **simple and user-friendly Streamlit interface**, making it easy for users to interact with the system.
- Delivers predictions in **real time**, providing instant results.
- The trained model can be **saved and reused** without retraining, improving efficiency.
- Can be further enhanced with advanced Machine Learning or Deep Learning algorithms to improve accuracy.
- Helps spread awareness about misinformation and encourages users to verify news before sharing.
- Serves as a practical example of applying **Artificial Intelligence** to solve real-world problems.
- Provides valuable hands-on experience in **data preprocessing, feature extraction, model training, and deployment**, making it an excellent learning project for students.

CONCLUSION

- The **Fake News Detection System** successfully demonstrates the practical application of **Machine Learning** and **Natural Language Processing (NLP)** for identifying fake and real news articles. By using **TF-IDF Vectorization** and the **Logistic Regression** algorithm, the system efficiently analyzes news text and provides accurate predictions through a simple and interactive Streamlit web application.
- The project achieved an accuracy of approximately **98.7%**, making it effective for text-based news classification. Throughout the development process, valuable concepts such as data preprocessing, feature extraction, model training, evaluation, and deployment were implemented, providing hands-on experience with real-world Machine Learning workflows.
- Overall, this project enhanced practical knowledge of Artificial Intelligence and Machine Learning while demonstrating how technology can be used to address the growing challenge of misinformation. With future improvements such as multilingual support, real-time fact-checking, and advanced Deep Learning models, the system can become even more accurate, scalable, and suitable for real-world applications.

FUTURE SCOPE

The **Fake News Detection System** can be further improved by incorporating advanced technologies and additional features to enhance its performance, usability, and real-world applicability.

- Integrate **Deep Learning models** such as **LSTM, BERT, or Transformer-based models** to improve prediction accuracy.
- Add **multilingual support** so the system can detect fake news in multiple languages.
- Connect the application with **live news APIs** to analyze and verify real-time news articles.
- Enhance the system by including **image and video analysis** to detect multimedia-based fake news.
- Develop a **mobile application** for easy access on smartphones and tablets.
- Deploy the application on **cloud platforms** such as AWS, Azure, or Google Cloud to improve scalability and accessibility.
- Incorporate **fact-checking APIs** to compare news articles with trusted sources for more reliable verification.
- Improve the user interface by adding features such as **news history, prediction confidence score, and user feedback**.
- Optimize the model using larger and more diverse datasets to improve performance on unseen news articles.
- Extend the system for use by **media organizations, educational institutions, social media platforms, and fact-checking agencies** to help reduce the spread of misinformation.

REFERENCES

The following resources were referred to during the development of the **Fake News Detection System**:

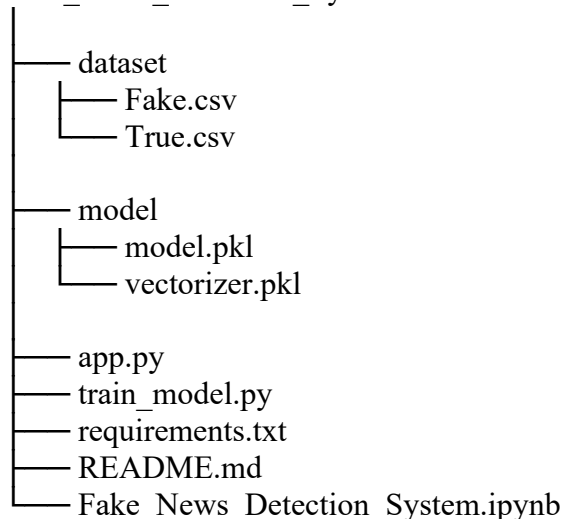
1. **Scikit-learn Documentation** – Machine Learning algorithms and utilities.
<https://scikit-learn.org/>
2. **Pandas Documentation** – Data manipulation and preprocessing.
<https://pandas.pydata.org/>
3. **NumPy Documentation** – Numerical computing in Python.
<https://numpy.org/>
4. **NLTK (Natural Language Toolkit) Documentation** – Natural Language Processing techniques.
<https://www.nltk.org/>
5. **Streamlit Documentation** – Development of the web application interface.
<https://streamlit.io/>
6. **Python Official Documentation** – Python programming language reference.
<https://docs.python.org/3/>
7. **Kaggle – Fake and Real News Dataset** – Dataset used for training and testing the model.
<https://www.kaggle.com/>
8. **GitHub** – Version control and project repository management.
<https://github.com/aartiprajapati-ai/>

APPENDIX

The appendix contains supporting materials and additional information related to the development of the Fake News Detection System

Appendix A – Project Folder Structure

Fake_News_Detection_System



Appendix B – Python Libraries Used

- Pandas
- NumPy
- Scikit-learn
- NLTK
- Streamlit
- Pickle

Appendix C-- Project Files

The project consists of the following files:

- **Fake.csv** – Fake news dataset
- **True.csv** – Real news dataset
- **train_model.py** – Model training script
- **app.py** – Streamlit web application
- **model.pkl** – Trained Machine Learning model
- **vectorizer.pkl** – Saved TF-IDF Vectorizer
- **requirements.txt** – Project dependencies
- **README.md** – Project documentation

Appendix D: Screenshots

The following screenshots are included as supporting evidence:

- Project Folder Structure
- Model Training Output
- Streamlit Home Page
- Fake News Prediction Result
- Real News Prediction Result

Appendix E: Acronyms

Abbreviation	Meaning
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
TF-IDF	Term Frequency–Inverse Document Frequency
CSV	Comma-Separated Values
UI	User Interface
IDE	Integrated Development Environment